

Sterownik: SC4-Hub zamiast ClearCore

Ustalenia z 2026-08-14. Dokument opisuje rozbieżność między sprzętem, który faktycznie jest na maszynie, a architekturą założoną w repozytorium — oraz proponowaną drogę wyjścia.

Problem

Repozytorium zakłada **Teknic ClearCore**: samodzielny sterownik z własnym firmware C++ (`firmware/clearcore/`), do którego serwer wysyła komendy wysokopoziomowe po **Ethernetcie** (TCP, port 8500). Interpolację ruchu, bazowanie i ciągły nadzór nad sygnałem zezwolenia wykonuje firmware.

Maszyna ma natomiast **Teknic SC4-Hub** z serwami **ClearPath-SC**.

Czym jest SC4-Hub

Hub komunikacyjno-I/O dla serw ClearPath-SC. Łączy **komputer** z silnikami po **USB** (alternatywnie RS-232) — do 4 silników na hub, do 4 hubów połączonych w łańcuch, czyli 16 osi na jednym porcie. Zasilanie 15–30 V.

Nie ma tam firmware do wgrania i nie ma Ethernetu. **Sterownikiem ruchu staje się PC**, przez bibliotekę Teknica **sFoundation** (C++, Windows/Linux; oficjalnych bindingów Pythona brak).

Wejścia/wyjścia huba:

- 2 wyjścia — nominalnie sterowanie hamulcami 24 V, ale wg SDK **mogą pracować jako wyjścia ogólnego przeznaczenia** (`BRAKE_0`, `BRAKE_1`),
- 1 wejście **Global Stop** — zatrzymuje wszystkie osie jednocześnie,
- każdy węzeł (silnik) udostępnia **2 wejścia ogólnego przeznaczenia** (A, B).

Źródła: teknica.com/sc4-hub, [instrukcja ClearPath-SC](#), `Beta_SDK_Examples/SC4-I/O/SC4-I-O.cpp` z pakietu sFoundation.

Korekta wcześniejszego ustalenia. W pierwszej analizie napisano, że SC4-Hub nie ma wyjścia nadającego się do sterowania wrzecionem. To nieprawda: przykład `SC4-I-O.cpp` opisuje oba wyjścia jako „*commonly used to control 24V Power Off Brakes, or as general purpose outputs*”. Jedno z nich może więc sterować przełącznikiem/stycznikiem wrzeciona 24 V — pozostaje sprawdzić obciążalność w specyfikacji huba.

Konsekwencje dla repozytorium

- `firmware/clearcore/` **jest bezużyteczne** — nie ma czego flashować. Cała logika ruchu musi przenieść się na PC.
- `clearCoreMachine` **nie zadziała w obecnej formie** — łączy się po TCP do `CLEARCORE_HOST:8500`

(server/app/machine.py).

- Serwa **muszą być serii SC**. ClearPath MC/SD (step & direction) SC4-Hub nie obsługuje.
Potwierdzone: na maszynie są serwa ClearPath-SC.

Bez zmian zostaje wszystko powyżej szwu `Machine`: panel operatora, edytor technologa, API MES, parser i walidator `.prg`.

Proponowana architektura: mostek sFoundation

Serwer rozmawia z maszyną wyłącznie przez klasę `Machine` (server/app/machine.py), a `ClearCoreMachine` tłumaczy program na **prosty protokół tekstowy** (PING, STATUS, HOME, MOVEXY, MOVEZ, JOG, SPINDLE, STOP, RESET — opis w `firmware/clearcore/README.md`; mostek dokłada RELEASE/HOLD i AXCFG).

Ten protokół jest gotowym szwem. Zamiast przepisywać warstwę Pythona, piszemy na PC **demoną C++ na sFoundation**, który wystawia *dokładnie ten sam* protokół na `127.0.0.1:8500`. Serwer uruchamiamy wtedy z:

```
MACHINE_MODE=clearcore CLEARCORE_HOST=127.0.0.1
```

i **nie zmieniamy ani linijki Pythona**. Istniejący `firmware/clearcore/main.cpp` przestaje być kodem do wgrywania, ale pozostaje **dobrą specyfikacją** tego, co mostek ma robić.

Ryzyka do rozstrzygnięcia

1. Bezpieczeństwo — założenie bez zmian

Wymóg z README (niezależny, certyfikowany układ sprzętowo odcinający zasilanie mocy serw) obowiązuje tak samo. Wejście **Global Stop w SC4-Hub to funkcja sterowania, nie funkcja bezpieczeństwa** — Teknic nie deklaruje dla niej kategorii bezpieczeństwa.

Przy PC jako sterowniku wymóg jest wręcz **ważniejszy** niż przy ClearCore: nierealtime'owy Linux na łączu USB oznacza, że zawieszenie procesu albo wypięcie kabla nie może zostawić osi w ruchu.

Stan: układ bezpieczeństwa na maszynie jest gotowy (potwierdzone przez użytkownika).

2. Interpolacja XY — otwarte

ClearCore robił interpolację w firmware. ClearPath-SC wykonuje własne profile **per oś**, komendowane po serialu; skoordynowany ruch po dowolnej linii z zadany posuwem nie jest tym, co ta magistrała robi natywnie.

Operacje **PUNKT** (zagłębienie w miejscu) są bezpieczne. Problem dotyczy operacji **LINIA**, gdzie liczy się tor — mostek może musieć dzielić linię na segmenty i pilnować synchronizacji osi. **Do zweryfikowania u Teknica przed wyborem architektury.**

3. Realtime i utrata łączności

Do zaprojektowania: zachowanie mostka przy utracie USB, zawieszeniu serwera i przy STOP w trakcie ruchu.

Pakiet sFoundation dla Linuksa

Linux_Software.tar.gz ze strony teknik.com/downloads (wydanie z 2026-08-13, ~1,9 MB). Zawiera źródła sFoundation, sterownik USB do SC4-Hub, dokumentację i przykłady.

Przykłady istotne dla nas:

- **Beta_SDK_Examples/SCNetworkReport** — wykrywa huby i raportuje widziane węzły. Nasz test rozpoznawczy.
- **Beta_SDK_Examples/SC4-IO** — wyjścia huba, Global Stop, wejścia węzłów.
- **SDK_Examples/Example-Homing, Example-Motion, Beta_SDK_Examples/MotionDualAxis** — bazowanie i ruch osi.

Pułapka: cdc_acm przechwyci hub

Z `ExarKernelDriver/driver_readme.txt`:

*Your system might have a built-in cdc_acm driver that will take control of the SC4-Hub. That driver is adequate for establishing communications with the board, but it is **NOT fully compatible with sFoundation**.*

Hub **wyenumeryuje się sam** jako `/dev/ttyACM0` i będzie wyglądał na działający, ale sFoundation nie będzie z nim poprawnie współpracować. Trzeba zainstalować dołączony sterownik Exar (wymaga roota i nagłówków jądra).

Uwaga: kernel ma w drzewie zarówno `cdc-acm.ko`, jak i `xr_serial.ko` — obie mogą przejąć urządzenie, więc kolejność ładowania ma znaczenie.

Stan środowiska (stacja robocza)

Ubuntu 22.04.5 LTS, kernel 6.8.0-40-generic. Teknic weryfikował procedurę m.in. na Ubuntu 24.04, które ma ten sam kernel.

Element	Stan
Nagłówki jądra (<code>/usr/src/linux-headers-6.8.0-40-generic</code>)	jest, <code>build</code> dowiązany
<code>xr_serial.ko</code> , <code>cdc-acm.ko</code> w drzewie	są, żaden niezaładowany
<code>g++ 11.4.0</code> , <code>make 4.3</code>	zainstalowane
Użytkownik w grupie <code>dialout</code>	tak (po <code>usermod</code> i restarcie)
<code>git</code>	brak (nieblokujące)

Gdzie leży SDK

vendor/teknic/ w repozytorium — katalog jest w `.gitignore`, bo to kod producenta pobierany z jego strony.

Pakiet trzymano początkowo w scratchpadzie sesji i **został skasowany między sesjami**.
Stąd trwała lokalizacja w projekcie.

- vendor/teknic/Linux_Software/ — rozpakowany pakiet (źródła, przykłady, sterownik USB, licencja).
- vendor/teknic/lib/ — instalacja single-user: `libsFoundation20.so` (854 KB), dowiązanie `libsFoundation20.so.1`, `MNuserDriver20.xml`.

Biblioteka zbudowana bez błędów (tylko ostrzeżenia `-Wwrite-strings` w `LibXML`). Przykłady buduje się z `make RPATH=<...>/vendor/teknic/lib`.

Instalacja sterownika Exar

Skrypt `Teknic_SC4Hub_USB_Driver/ExarKernelDriver/Install_DRV_SCRIPT.sh` (już ustawiony jako wykonywalny). Wymaga **roota i prawdziwego terminala** — w trakcie działania czeka na naciśnięcie klawisza. Co robi:

1. Buduje `xr_usb_serial_common.ko` przeciw bieżącemu jądro i kopiuje do `/lib/modules/$(uname -r)/kernel/drivers/usb/serial`, `depmod -a`, `insmod`.
2. Dopisuje `xr_usb_serial_common` do `/etc/modules`, czyli **moduł wraca po restarcie**.
3. Prosi o podłączenie huba, po czym sam **odpina go od `cdc_acm`** i przypina do `cdc_xr_usb_serial`.

Uruchomienie (z katalogu skryptu — skrypt to sprawdza):

```
cd vendor/teknic/Linux_Software/Teknic_SC4Hub_USB_Driver/ExarKernelDriver
sudo ./Install_DRV_SCRIPT.sh
```

SC4-Hub ma USB ID 2890:0213 — po tym rozpoznajemy go w `lsusb` i `/sys/bus/usb/devices/*/modalias` (skrypt szuka wzorca `v2890p0213`).

Wymagany gcc-12

Moduł jądra musi być zbudowany **tym samym kompilatorem co jądro**. Kernel 6.8.0-40 (HWE) zbudowano `gcc-12`, a `build-essential` na 22.04 daje `gcc-11` — skrypt przerywa się na `gcc-12: not found`.
Rozwiązanie:

```
sudo apt install -y gcc-12
```

`gcc-12` instaluje się obok `gcc-11` i nie podmienia domyślnego kompilatora. Nie trzeba podawać `CC=` — `Makefile` jądra sam sięga po właściwą wersję.

Uwaga: skrypt trzeba powtórzyć **po każdej aktualizacji jądra** — moduł jest wiązany z konkretną wersją jądra i jej kompilatorem, DKMS tu nie ma.

Plan rozpoznania

1. ~~apt install build-essential + usermod + a6 dialout +~~ przełogowanie: **zrobione**
2. Budowa ~~libsFoundation20.so~~, instalacja ~~single-user~~: **zrobione**
3. Budowa narzędzia ~~SCNetworkReport~~: **zrobione**, linkuje się poprawnie (ldd wskazuje na `vendor/teknic/lib`).
4. Instalacja sterownika Exar: **zrobione** (po doinstalowaniu `gcc-12`).
5. Podłączenie SC4 Hub po USB: **zrobione**
6. Weryfikacja enumeracji i przypięcia sterownika: **zrobione**
7. Uruchomienie ~~SCNetworkReport~~: **zrobione** — **3 węzły wykryte**.

Wynik rozpoznania (2026-08-14)

Warstwa USB — działa w całości

To była najbardziej ryzykowna część i jest zamknięta:

```
Bus 002 Device 007: ID 2890:0213 Teknic, Inc ClearPath 4-axis Comm Hub
```

- oba interfejsy (2-1.2:1.0, 2-1.2:1.1) przypięte do `cdc_xr_usb_serial`, **nie** do `cdc_acm` — czyli sterownik Exar zadziałał zgodnie z ostrzeżeniem z `driver_readme.txt`,
- moduł `xr_usb_serial_common` załadowany,
- port `/dev/ttyXRUSB0`, grupa `dialout`, użytkownik ma dostęp.

Pułapka: `cdc_acm` wraca przy każdej enumeracji

Skrypt Teknica przepina hub na sterownik Exar **jednorazowo**. Po każdym resecie huba (włączenie 24 V, przewtyknięcie kabla, restart szafy) urządzenie enumuje się od nowa i `cdc_acm` przejmie je z powrotem — pojawia się `/dev/ttyACM0` zamiast `/dev/ttyXRUSB0`, a sFoundation zgłasza „No SC4-HUB's found”.

Rozwiązanie: `tools/sc4hub-rebind.sh` (ręcznie) i `tools/99-teknic-sc4hub.rules` (automatycznie, przez `udev`). Szczegóły: [zmiany/narzedzia-usb-sc4hub.md](#).

Sieć silników — działa

Po zasileniu huba (24 V) **oraz** magistrali DC silników i po przełączeniu sterownika, `SCNetworkReport` widzi komplet osi:

```
Node[0]  CPM-SCSK-2310S-RLNA-1-8-D   S/N 90404362   FW 1.8.0 EEFF
Node[1]  CPM-SCSK-2310S-RLNA-1-8-D   S/N 90406231   FW 1.8.0 EEFF
Node[2]  CPM-SCSK-2310S-RLNA-1-8-D   S/N 90406002   FW 1.8.0 EEFF
```

Trzy identyczne serwa SCSK (NEMA 23), pierścień zamknięty, komunikacja end-to-end od Pythona przez sFoundation do silników. **Droga mostka sFoundation jest potwierdzona jako wykonalna.**

Wcześniejsze błędy i ich przyczyny:

Objaw	Przyczyna
Port failed to initialize, err=0x80040601	wyłączone 24 V huba i brak zasilania silników — pętla prądowa przerwana
No SC4-HUB's found przy sprawnym USB	cdc_acm przejął hub po ponownej enumeracji

Potwierdzone: pełny cykl na sprzęcie

Mostek (bridge/, patrz [zmiany/mostek-sc4hub.md](#)) przejechał **cały program 583912004711 na trzech serwach**: wybór zlecenia w MES → bazowanie → operacje 1–3 → PAUZA → wznowienie → powrót do zera. Pozycje trafione co do impulsu, wrzeczono przełączane zgodnie z programem. Serwer Pythona nie wymagał żadnej zmiany w logice — protokół tekstowy zadziałał jako szew dokładnie tak, jak zakładano.

Jednostki osi

Odczytane z serwa: **800 imp/obr** (Info.PositioningResolution). Śruba kulowa: **5 mm/obr**. Razem **160 imp/mm**.

Od czasu wprowadzenia ekranu konfiguracji osi mm/obr jest ustawiane **osobno dla każdej osi** i przysyłane przez serwer komendą AXCFG (patrz [konfiguracja-osi.md](#)); MM_PER_REV w machine.env jest już tylko wartością startową mostka.

Wpisanie tej rozdzielczości na sztywno (błędnie 6400) dawało ruch 8× wolniejszy przy pozornie poprawnym odczycie pozycji — błąd skracał się w przeliczeniu tam i z powrotem. Dlatego mostek czyta ją ze sprzętu, a nie z konfiguracji.

Mapowanie osi — ustalone

Ustalone trybem --identify i utrwalone w bridge/machine.env **po numerach seryjnych**, żeby przełożenie wtyków go nie zmieniło:

Oś	Węzeł	S/N	Rola
Z	Node[0]	90404362	oś frezu
X	Node[1]	90406231	pozioma
Y	Node[2]	90406002	pionowa

Kolejność w pierścieniu **nie** odpowiada kolejności osi — domyślne mapowanie 0 → X, 1 → Y, 2 → Z byłoby błędne.

Auto-Tune: kiedy blokuje, a kiedy nie

Pole `userID` wszystkich trzech węzłów to **Unloaded** — to fabryczna konfiguracja. Zgodnie z dokumentacją Teknica serwa ClearPath-SC są „*pre-configured for **unloaded** use only*” i **wymagają uruchomienia Auto-Tune po sprzężeniu z mechaniką**.

Auto-Tune jest funkcją **ClearView**, który działa tylko na **Windows** — na Linuksie nie ma odpowiednika. Konsekwencja:

- **potrzebny jednorazowo komputer z Windows** (albo maszyna wirtualna z przekazaniem USB), żeby zestroić każdą oś pod jej rzeczywiste obciążenie i zapisać konfigurację (`File` → `Save Configuration`, plik `.mtr`),
- **wczytanie gotowego `.mtr` działa już z Linuksa** przez `sFoundation` — przykład `Beta_SDK_Examples>LoadingConfigFile`.

Konfiguracja fabryczna jest właściwa dla serw bez mechaniki — i w takim stanie przejechano cały cykl. Blokada dotyczy wyłącznie pracy pod obciążeniem: po sprzężeniu z mechaniką **nie należy uruchamiać osi przed Auto-Tune**.

Źródło: [instrukcja ClearPath-SC](#), `Beta_SDK_Examples>LoadingConfigFile>LoadingConfigFile.cpp`.

Do sprawdzenia: E-stop a komunikacja

Jeśli układ bezpieczeństwa odcina magistralę DC silników, to wciśnięty E-stop **zabija także komunikację** — maszyna nie tylko staje, ale przestaje widzieć osie. Mostek musi odróżniać ten stan od awarii łącza i wracać do pracy po zwolnieniu E-stopu. Test: przebieg `SCNetworkReport` z wciśniętym E-stopem.

Uwaga na przyszłość: diagnostyka pętli

Gdyby port znowu przestał się otwierać (`err=0x80040601`), kolejność sprawdzania: 24 V huba (dioda) → zasilanie magistrali DC silników → czerwona zworka „end-of-loop” (zamyka pierścień, położenie zależy od liczby silników) → obsadzenie złącz sekwencyjnie od pierwszego, bez przerw.

Źródło: [Troubleshooting MSP/ClearView Communication](#).

Do zrobienia

Przed zabudową mechaniki:

- ☐ **Auto-Tune każdej osi pod obciążeniem** — wymaga Windows z ClearView; zapisać `.mtr` i wczytywać z Linuksa (`LoadingConfigFile`).
- ☐ **Skonfigurować homing w ClearView** — dziś bazowanie jest tylko zerowaniem programowym, co na maszynie z mechaniką nie wystarczy.
- ☐ Zweryfikować pomiarowo tor operacji **LINIA** (interpolacja przybliżona).

Testy, których jeszcze nie zrobiono:

- **STOP w trakcie ruchu** — zatrzymanie w 30 ms, maszyna staje, **ALARM** z komunikatem. Patrz

[zmiany/status-i-zatrzymanie.md](#).

- ☐ Utrata zezwolenia w trakcie ruchu — wymaga fizycznego zadziałania układu bezpieczeństwa.
- ☐ Zachowanie komunikacji przy wciśniętym E-stopie (czy odcina magistralę DC).
- ☐ Reguła udev (`tools/99-teknic-sc4hub.rules`) — instalacja i weryfikacja przez przewtyknięcie huba.

Pozostałe:

- ☐ Obciążalność wyjść `BRAKE_0/BRAKE_1` pod stycznik wrzeczona.
- ☐ Zdecydować o losie `firmware/clearcore/` — usunąć czy zostawić jako specyfikację protokołu (mostek go implementuje).
- ~~Przenieść pakiet Teknica do `vendor/teknic/` (poza gitem).~~
- ~~Ustalić mapowanie węzłów na osie.~~
- ~~Potwierdzić wykonalność mostka sFoundation.~~

Źródło: `docs/sterownik-sc4-hub.md` — maszyna do ocinania wlewków płytek optyki